

FieldGPU: A GPU-based Version of Field II with Python Bindings for Large Scale Simulations and Complex Transducer Configurations

Florence Klitzner, Rüdiger Göbl*, Christoph Hennemersperger, Stefan Wörz

LUMA Vision, Beech Hill Road, Dublin, Ireland

*Now with ImFusion GmbH, Munich, Germany

{florence.klitzner, r.goeb1}@tum.de, stefan.woerz@lumavision.com

Abstract—We present FieldGPU, a CUDA-accelerated re-implementation of the Field II ultrasound simulation framework, accessible as a Python package. FieldGPU leverages GPU parallelism to improve scalability and reduce simulation times for large transducer arrays and high-quality simulation scenarios. It supports most core Field II features, including transmit and receive simulation and custom transducer configurations with rectangular aperture elements. Comparison benchmarks performed between Field II and FieldGPU demonstrate that FieldGPU achieves up to 1100 times performance increase compared to Field II for the largest tested simulation with 10 million scatterers. Our simulation method significantly reduces simulation times and provides a modern Python interface that facilitates the direct translation of Field II simulation scripts to FieldGPU.

Index Terms—FieldII, FieldGPU, Ultrasound Simulation, GPU Computing, CUDA, Python

I. INTRODUCTION

Accurate ultrasound (US) imaging simulations enable the modelling and analysis of complex interactions between US waves and biological tissues without requiring extensive experimental setups. Several simulation frameworks exist to address this task, with Field II [1] often considered the gold standard baseline for comparison to others, due to its broad adoption, comprehensive feature set, and precision.

This work focuses on Field II's model, which simulates US imaging by calculating the emitted and received acoustic fields based on user-defined transducer configurations and tissue properties described through individual scatterers. The core simulation computes the spatial impulse response (SIR) and describes how the US wave propagates through a medium and interacts with these scatterers. By modelling both the transmit and receive processes, Field II can generate pulse-echo data that accurately reflects the response of the simulated US system [2]. Despite its strengths, the performance of Field II suffers in scenarios with complex transducer geometries or large amounts of scatterers. The primary reason for this is that Field II operates as a single-threaded MATLAB toolbox, and although a commercial version exists offering multi-core processing and Python bindings [3], applying parallel processing with modern GPUs to the Field model could significantly improve simulation times even on lower-end hardware while making large-scale simulations more accessible.

We introduce FieldGPU, which builds upon the foundation of Field II by re-implementing its core functionalities while utilizing the advantages of GPU computations with CUDA [4]. It preserves the accuracy and flexibility of the original platform, keeping the skills for users transferable, and improves the accessibility of the underlying methods to enable higher customization for simulation setups. By providing a Python-based interface, we ensure compatibility with modern workflows, providing seamless integration into existing US and machine learning pipelines.

The following sections present FieldGPU's simulation method, discuss its limitations, and outline its use of modern GPU parallelization. Performance is evaluated against Field II across a range of simulation scenarios, emphasizing computation time and scalability improvements. A final comparison is made on a diverging wave imaging case between the two versions.

II. RELATED WORK

A number of other frameworks and approaches have been developed alongside Field II. These platforms for ultrasound can be differentiated using several properties, with the most relevant being whether they operate in the time or frequency domain, use a linear or non-linear acoustic model, and use mesh-based or mesh-free methods. Linear models assume a consistent waveform shape, which is valid for typical diagnostic imaging, while non-linear models can capture additional harmonic generation or distortions of such waveforms [5]. Mesh-free models like Field II approximate scatterers directly in a continuous space [6], while mesh-based models like K-wave discretize the medium into a grid, allowing more heterogeneous tissue to be simulated [7].

Field II operates in the time domain and is limited to linear acoustics as well as homogeneous media, making it well-suited for simulating imaging point spread functions. In contrast, K-wave, its most widely adopted alternative, supports a broader non-linear model in both the harmonic and time domains, enabling modelling of more complex tissue interaction. Part II of introducing the SIMUS simulation platform provides a more comprehensive comparison between current methods [5].

Ray-tracing based methods have also been developed, like the work presented by Kutter et al., which uses CT-derived

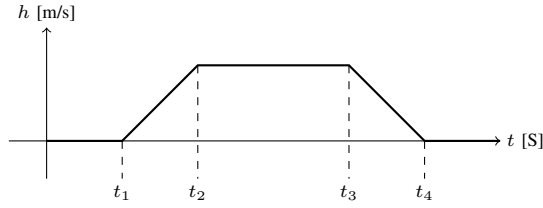


Fig. 1. Description of Far-field response (recreated from [6]).

volumes and achieves near real-time performance in a GPU-based pipeline, demonstrating the significant potential of large scale parallel processing on the GPU for ultrasound simulations [8].

In contrast to these broader approaches, our work builds directly on Field II's established model and focuses on re-implementing its simulation method with GPU acceleration, which we detail in the following section.

III. SIMULATION METHOD

A. Calculation of the Spatial Impulse Response (SIR)

FieldGPU simulates the pulse-echo ultrasound response by modelling wave transmission and echo reception from point scatterers in tissue. An ultrasound source, like a transducer consisting of multiple elements with a specific shape and size, is defined by its aperture, the active area of the transducer that emits or receives sound waves. The core of Field II and FieldGPU's approach is to compute the Spatial Impulse Response (SIR), describing how a single element's impulse propagates through the medium and contributes to the echo. The full Radio-frequency (RF) echo signal can be predicted by summing contributions from all elements and scatterers.

FieldGPU implements the far-field SIR formulation for rectangular elements, as presented by Jensen and Svendsen in Eqs. 13–15 [6]. For an element of size $w_x \times w_y$ at a field point at distance l away, the impulse response has a trapezoidal time profile with breakpoints $t_1 \dots t_4$ (arrival times of the element's extreme edges) as shown in Fig. 1. The amplitude A and the slope m depend on the piston area, distance to the field point, and arrival times:

$$A = \frac{w_x w_y}{2\pi l(t_3 - t_1)}, \quad m = \frac{A}{2(t_2 - t_1)} \quad (1)$$

The piecewise SIR as a function is then given as:

$$h(t) = \begin{cases} m(t - t_1) & t_1 \leq t < t_2 \\ A & t_2 \leq t < t_3 \\ m(t_4 - t) & t_3 \leq t < t_4 \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

For high sampling rates $f_s \gg 1/\min\{t_{i+1} - t_i\}$, simple point sampling of $h(t)$ adequately captures the trapezoidal shape. In contrast, at lower sampling rates, such point-sampling can be insufficient, misrepresenting the shape of the far-field response. To maintain accuracy across different sampling rates, we calculate the function's definite integral between

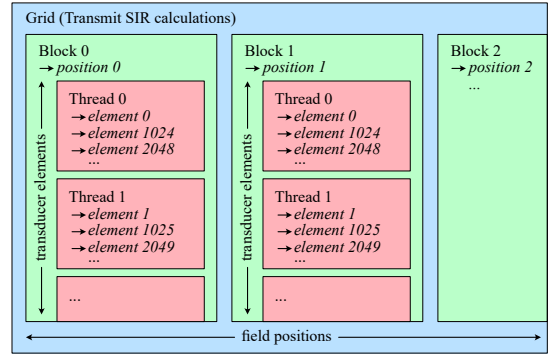


Fig. 2. Overview of how transmit SIR calculations are distributed for calculating the induced pressure field

$t \pm 0.5 * \Delta t$. The main advantage of this approach is its simplicity, as it can be computed quickly and yields acceptable results even for low sampling rates.

B. Limitations

However, this approach is only valid in the far-field where the SIR simplifies as the wavefronts on the element are nearly parallel. In the near-field, they are curved and require a modified method [6], which FieldGPU does not implement yet. To enable accurate simulation in the near-field, single elements are subdivided into multiple mathematical elements. The distance l of the far-field region is defined by the largest dimension of the rectangular element w and the wavelength size λ [6]:

$$l \gg \frac{w^2}{4\lambda} \quad (3)$$

As the distance of the far-field region is directly dependent on the element's size, using multiple smaller elements to represent a single larger one effectively moves the far-field region closer to the element surface. In FieldGPU, elements are subdivided into multiple smaller mathematical elements, enabling accurate simulation in the near-field. As Field II uses a different method for evaluating such close-range SIRs, each platform will show slight differences when comparing their near-field accuracy.

C. Distribution across parallel GPU threads

FieldGPU implements the main simulation functions from Field II, namely `calc_scatter`, `calc_scatter_multi`, and `calc_scatter_all`, which compute received voltage signals for the entire transducer, for each receiving element, and for each transmit-receive element pair, respectively; as well as `calc_h` for spatial impulse responses (SIRs), `calc_hp` for induced pressure fields, and `calc_hhp` for echo sensitivity fields. Each of these require partially different approaches on how they are executed efficiently on a GPU. Generally, a CUDA kernel consists of a grid of blocks, composed of threads executing in parallel [4]. Fig. 2 illustrates how the calculation of the transmit SIRs for the function `calc_h` is distributed across such a kernel. For all given 3D positions, at which we want to measure the induced pressure field, blocks are defined, which then in turn assign one thread for each

TABLE I
SIMULATION PARAMETERS USED FOR THE PERFORMANCE ANALYSIS
AND IMAGE COMPARISON (FROM [3]).

Parameter	Symbol / Value	Unit
Sampling frequency	$f_s = 100$	MHz
Speed of sound	$c = 1540$	m/s
Transducer center frequency	$f_0 = 5$	MHz
Number of cycles in pulse	$M = 2$	
Wavelength lambda	$\lambda = c/f_0 = 0.308$	mm
Width of element	$\lambda/2 = 0.3$	mm
Height of element	$h = 5$	mm
Kerf	$k_e = \lambda/10 = 0.031$	mm
Fixed focal point	$f_c \in [0, 0, 60]$	mm
Number of physical elements	$N_e = 64$	
Pulse duration	$T_p = M/f_0 = 0.4$	μs
Virtual Source Distance (for Fig. 5)	$D_{v_source} = 8.54$	mm

transducer element's calculations. CUDA threads per 1D block are limited to 1024, so we process multiple elements per thread for large transducers.

Otherwise FieldGPU mainly uses 2D blocks when processing elements and their sub-elements in parallel. Both 2D and 3D grids are used to calculate and convolve received voltage signals, defining each combination of transmit and receive elements and position.

IV. RESULTS

To evaluate FieldGPU's performance, we compare benchmarks with the original Field II implementation across varying scatterer amounts, from 100 to 10^7 scatterers, over 100 steps spaced evenly on a log scale. The system used for performance tests features a NVIDIA RTX 3090, an AMD Threadripper 3970X, and 128 GB of RAM. Our Field II baseline is run locally on Ubuntu 20.04 using Octave [9]. Additionally, we also compare with the measured execution time of running Field II through Matlab Online [10]. The former is referred to as the "Local" and the latter as the "Web" run of Field II.

Table I shows our simulation parameters, which follow the ones used by Jensen to evaluate Field II Pro [3]. We did not have access to Field II Pro and could not directly compare it to our implementation. In the performance tests, Gaussian distributed scatterers are placed as a plane along a depth ranging from 5 to 55 mm and a lateral range from -10 to 10 mm. No dynamic focusing or apodization is used in the simulation.

A. Performance

Fig. 3 shows the execution time of the `calc_scatter` function, calculating the received signal for a transducer from a collection of scatterers. FieldGPU completes the largest scenario (10 million scatterers) in 1 second, compared to 18 minutes and 33 seconds for the Field II version run in the online version of Matlab, corresponding to a speed-up of approximately 1100 times faster for FieldGPU.

More recent performance tests for Field II Pro by Jensen show a simulation time of 12 minutes and 30 seconds for Field II's web version and 5.8 seconds for Field II Pro for 10 million scatterers using 64 Threads on an AMD Ryzen Threadripper

2990WX for a total speed-up of 129 [11]. While we could not recreate these results in our performance tests, with our baseline performance for Field II being lower, FieldGPU still achieves a higher overall speed-up than previous multi-threaded optimizations.

An overview of the relative speed-ups for the functions `calc_scatter` and `calc_scatter_multi` is given in Fig. 4. Generally the performance gain increases for higher amounts of scatterers utilizing the advantages of the parallel GPU processing more. The function `calc_scatter_multi` achieves a speed-up of 1370 times faster for the largest scenario, completing it in 4.3 seconds with FieldGPU and 94 minutes and 30 seconds with the web version of Field II. Generally an average speed-up achieve of 655 for `calc_scatter` and 668 for `calc_scatter_multi` is achieved comparing all scenarios.

B. Image Comparison

We compare beamformed B-mode images generated by Field II and FieldGPU for a scenario using a singular diverging wave (DW) transmit event without steering. The transducer configuration is given in Table I, and the diverging wave is generated using transmit delays. The simulated phantom includes: (1) Ten point targets along the depth axis in 10mm steps, (2) five lateral point targets from -20mm to 20mm at a 40mm depth, and (3) a 10mm cyst at 30mm depth. The scatterer density is 500 scatterers/mm² for a total of 6,000,000 scatterers in a 120 x 100 mm² phantom.

As shown in Fig. 5, slight discrepancies accumulate in the near-field and side lobes of the image, but FieldGPU produces results visually comparable to those of Field II, demonstrating the accuracy of our GPU-based implementation.

V. DISCUSSION AND CONCLUSION

High-quality simulations of US imaging scenarios can present a significant time cost in development cycles and data generation tasks. Previous implementations of the field model have relied on CPU computation. While the multi-core capabilities of modern processors achieved significant speed-ups [3], we demonstrate further enhancements using the massive parallel processing capabilities of modern GPUs.

Our results show that FieldGPU consistently achieves average speed-ups of over 600 times faster compared to Field II, for both tested functions, and up to 1370 times faster for the largest tested scenario. FieldGPU's architecture accelerates computation and scales effectively across simulation sizes, supporting scenarios that are impractical with CPU-based methods, and demonstrates visual equivalence to Field II's outputs with minor deviations in the near-field regions.

While FieldGPU's feature set does not have full parity with Field II's, and only supports rectangular elements, most typical simulation scenarios can be directly translated to FieldGPU while offering the advantages of a Python-native interface. This enables seamless integration with data pipelines, machine learning workflows, and post-processing libraries. FieldGPU offers a significant performance and usability advancement, providing both speed and flexibility, while preserving compatibility with established workflows.

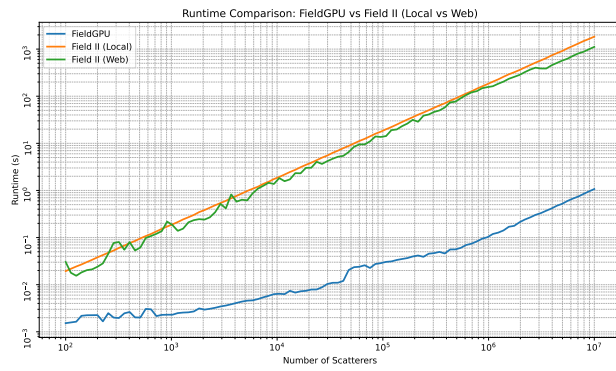


Fig. 3. Runtime comparison between FieldGPU and Field II for both web and local runs and the function `calc_scatter`.

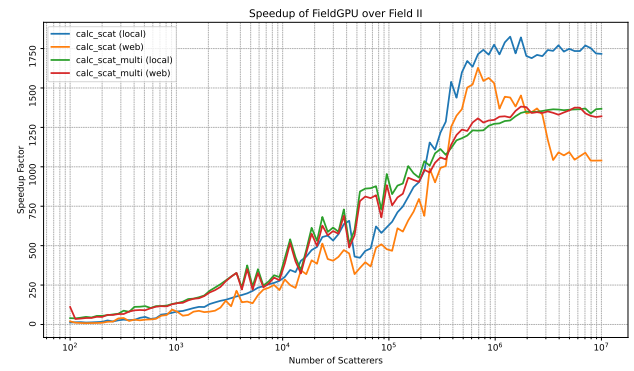


Fig. 4. Speedup between FieldGPU and Field II for both web and local runs and the functions `calc_scatter` and `calc_scatter_multi`.

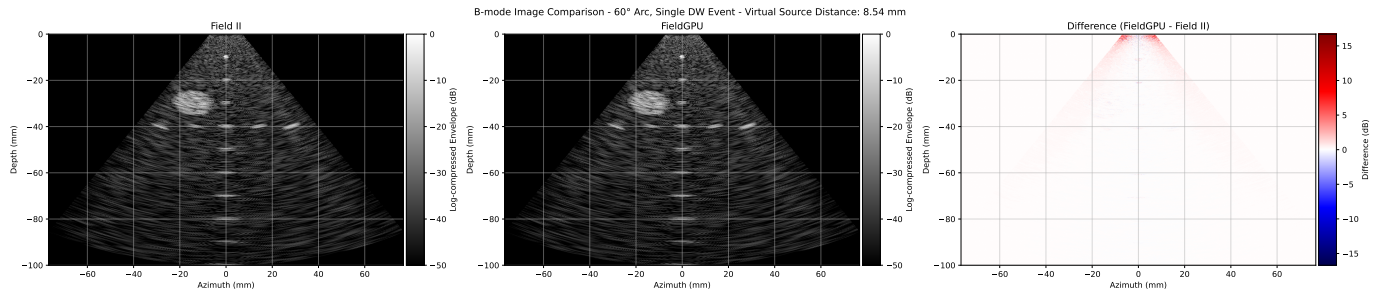


Fig. 5. Comparison of B-mode images from Field II (left) and FieldGPU (middle) simulations from a singular diverging wave transmit event with no steering; right image shows their difference.

REFERENCES

- [1] J. A. Jensen, "Field: A program for simulating ultrasound systems," *Medical & Biological Engineering & Computing*, vol. 34, no. sup. 1, pp. 351–353, 1997.
- [2] —, "Simulation of advanced ultrasound systems using field ii," in *2004 2nd IEEE International Symposium on Biomedical Imaging: Nano to Macro (IEEE Cat No. 04EX821)*. IEEE, 2004, pp. 636–639.
- [3] —, "A multi-threaded version of field ii," in *2014 IEEE International Ultrasonics Symposium*. IEEE, 2014, pp. 2229–2232.
- [4] J. Nickolls, I. Buck, M. Garland, and K. Skadron, "Scalable parallel programming with cuda: Is cuda the parallel programming model that application developers have been waiting for?" *Queue*, vol. 6, no. 2, pp. 40–53, 2008.
- [5] A. Cigier, F. Varray, and D. Garcia, "Simus: An open-source simulator for medical ultrasound imaging. part ii: Comparison with four simulators," *Computer Methods and Programs in Biomedicine*, vol. 220, p. 106774, 2022.
- [6] J. A. Jensen and N. B. Svendsen, "Calculation of pressure fields from arbitrarily shaped, apodized, and excited ultrasound transducers," *IEEE transactions on ultrasonics, ferroelectrics, and frequency control*, vol. 39, no. 2, pp. 262–267, 1992.
- [7] B. E. Treeby and B. T. Cox, "k-wave: Matlab toolbox for the simulation and reconstruction of photoacoustic wave fields," *Journal of biomedical optics*, vol. 15, no. 2, pp. 021 314–021 314, 2010.
- [8] O. Kutter, R. Shams, and N. Navab, "Visualization and gpu-accelerated simulation of medical ultrasound from ct images," *Computer methods and programs in biomedicine*, vol. 94, no. 3, pp. 250–266, 2009.
- [9] J. W. Eaton, D. Bateman, S. Hauberg, and R. Wehbring, *GNU Octave version 5.2.0 manual: a high-level interactive language for numerical computations*, 2020, accessed: Aug. 1, 2025. [Online]. Available: <https://www.gnu.org/software/octave/doc/v5.2.0/>
- [10] The MathWorks, Inc. (2025) Matlab version: 25.1.0 (r2025a). Accessed: Aug. 13, 2025. [Online]. Available: <https://www.mathworks.com>
- [11] J. A. Jensen. (2025) Field II Pro – Performance. Accessed: Aug. 13, 2025. [Online]. Available: https://field-ii.dk/?field_ii_pro/field_ii_pro_main.html